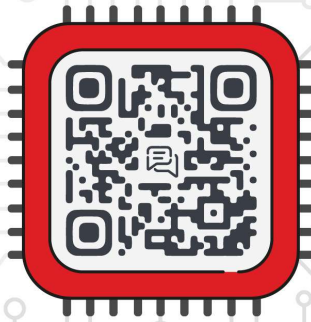




7

6

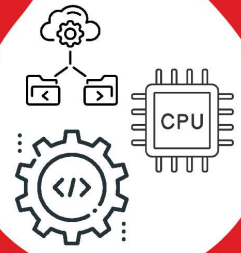
1050 sp

نظري

كلية الهندسة المعلوماتية

السنة الرابعة

البنيان المؤسساتي



د. عمار لامع جوخدار محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs; 04/05/2026

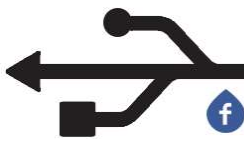
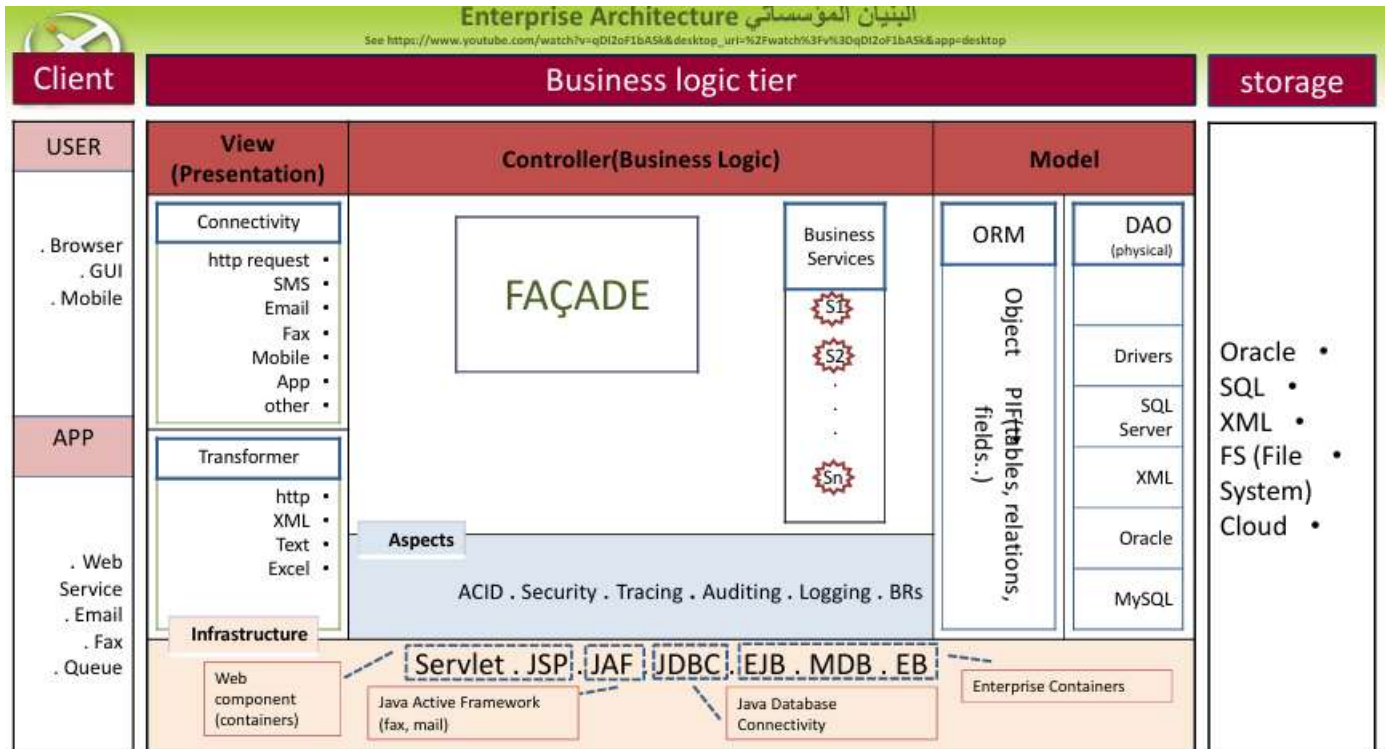
برمجة تفرعية

البنيان المؤسساتي

Enterprise architecture

سبق و تحدثنا عن البنيان المؤسساتي و اليوم سنغوص في أعماقه لنكتشف كيف يعمل.

- ملاحظة: Design Patterns هي حلول يستخدمها الإنسان للمشاكل المكررة و هي ليست كود جاهز و إنما تختلف من حالة لأخرى
- كثرة الموظفين بالشركة لا يعني أنها تعمل على التوازي و أنها شركة ضخمة و مهمة فغالبا يكون 20٪ من الشركة للدعم الفني للرد على المستخدمين.



البنية التحتية Infrastructue و من مكوناتها:

- Servlet: هو وسيط بين المستخدم و السيرفر لتوليد المحتوى الديناميكي على الإنترنت
- JSP (web component): تقنية تجمع بين HTML و كود جافا لتوليد صفحات تفاعلية تتغير حسب البيانات أو طلبات المستخدم.
- JAF (Java Active Framework): إطار عمل يساعد على تحديد نوع البيانات العشوائية، و تغليف الوصول إليها، و اكتشاف العمليات المتاحة و ينشئ الكائن المناسب للتعامل معها.
- JDBC (Java Database Connectivity): واجهة برمجية تسمح لتطبيقات جافا بالتواصل مع قواعد البيانات.
- EJB (Enterprise JavaBeans): تقنية من جافا تستخدم لتطوير تطبيقات المؤسسات الكبيرة و المعقدة، و تساعد على بناء مكونات برمجية صغيرة (Beans) قابلة لإعادة الاستخدام، و تقدم خدمات مثل إدارة المعاملات، الأمان، المكالمات عن بعد، و غيرها.
- MDB (Message-Driven Beans): نوع خاص داخل ال EJB مخصص للتعامل مع الرسائل غير المتزامنة مثل معالجة الرسائل الواردة في الخلفية بدون أن يتوجب على المستخدم الانتظار.
- EB (Enterprise Containers): مصطلح عام يشمل كل أنواع ال Beans التي تبنيها باستخدام EJB.

سابق و تحدثنا أن بنية البنيان المؤسساتي تتكون من 3 طبقات أساسية:

1. طبقة ال client : تتألف من:

1. user، مثل:

browser - gui - mobile

2. app، مثل:

web service - email - fax - queue

2. طبقة ال storage

مثل : Oracle , sql, xml, fs, cloud

3. طبقة ال business logic:

و هي تمثل طبقة العمل (know how) أي في حال عمل تطبيقين لجامعتين مختلفتين التغيير سيكون في طبقة التخزين (storage) في قاعدة البيانات (database) أما منطق العمل سيبقى نفسه.

سؤال مهم: من هو الشخص الذي يتحكم بالمرونة (flexibility) في المشروع؟

الجواب: المحلل

ملاحظة: في المحاضرات القادمة سنتطرق إلى مفهوم جديد اسمه (data clensing) تنقية البيانات و هي خطوة أساسية لاكتشاف و تصحيح أو حذف البيانات الخاطئة أو المكررة أو غير المكتملة في database أي تنظيفها قبل إدخالها للتطبيق.

- طبقة ال client تقوم بإرسال البيانات إلى طبقة ال business logic و تكون إما App أو زبون
- ملاحظة: المخ ليس له علاقة بالبيانات، في حال كان سليم و البيانات سليمة سيعمل بكفاءة عالية، نقصد بالمخ هو طبقة ال Logic ، أما في حال أحدهما به أخطاء فالخرج لن يكون مثالي.

طبقة ال Business Logic تتكون من ثلاث طبقات أساسية:

MVC:

1. view (presentation)

الطبقة التي تتخاطب مع الآخرين و تأخذ البيانات و ترجع الخرج.

ضمن طبقة ال presentation يوجد لدينا Connectivity تستقبل الرسالة من الأشكال :
Http Request - sms - email - fax ، و ترسلها إلى Transformer الذي يقوم بدوره بتحويلها إلى شكل واحد من الأشكال:

(http - xml - text - excel)

نطلق عليها Message حتى يتم التعامل مع البيانات بسهولة لاحقاً.

2. Controller

تحلل البيانات (تعالجها).

3. Model

تحويل البيانات إلى Model ثابت حتى أستطيع التعامل معه لاحقاً.

سؤال مهم: لماذا تصف ال message على أنها serializable؟

لأنه يمكن إرسالها على الشبكة و تخزينها، لذلك أصبح من الأسهل تمرير باراميترات التوابع عن طريق Message.

فائدة أريسيبزية:

ماذا يعني Serializable ؟

هو قدرة كائن (Object) على التحول إلى شكل يمكن تخزينه أو نقله، مثل تحويله إلى سلسلة من البايتات (bytes) بحيث يمكن إعادة بناء هذا الكائن لاحقاً بنفس البيانات

مثال:

في حال لدينا التابع التالي $f(a,b)$ و ضمن التابع نقوم بتمرير a,b لتابع آخر اسمه g ، بعد تطبيق بعض العمليات الحسابية على a, b في حال أصبح g يأخذ متحول ثالث و يجب تمريره إلى f أيضاً، فيجب علينا عندئذ تعديل الكود كاملاً.
أما في حال كان $f(\text{message})$ التابع f يستقبل رسالة و ليس متحولات و يمررها إلى g ، عندها لن نحتاج لتعديل الكود كاملاً في حال تغيير جزء من الرسالة.

سؤال مهم: من أهم الوثوقية في البرامج (Reliability)؟

صغر حجم الكود (عدد أسطر أقل).

■ ال client لا يرى الخدمة و إنما يستدعي ال facade، و هي تعني المظهر الخارجي أو الواجهة التي تقوم بدورها باختيار الخدمة المناسبة، و ال facade هو design pattern

سؤال مهم: ماذا تستقبل طبقة ال presentation؟

تستقبل فقط Api.

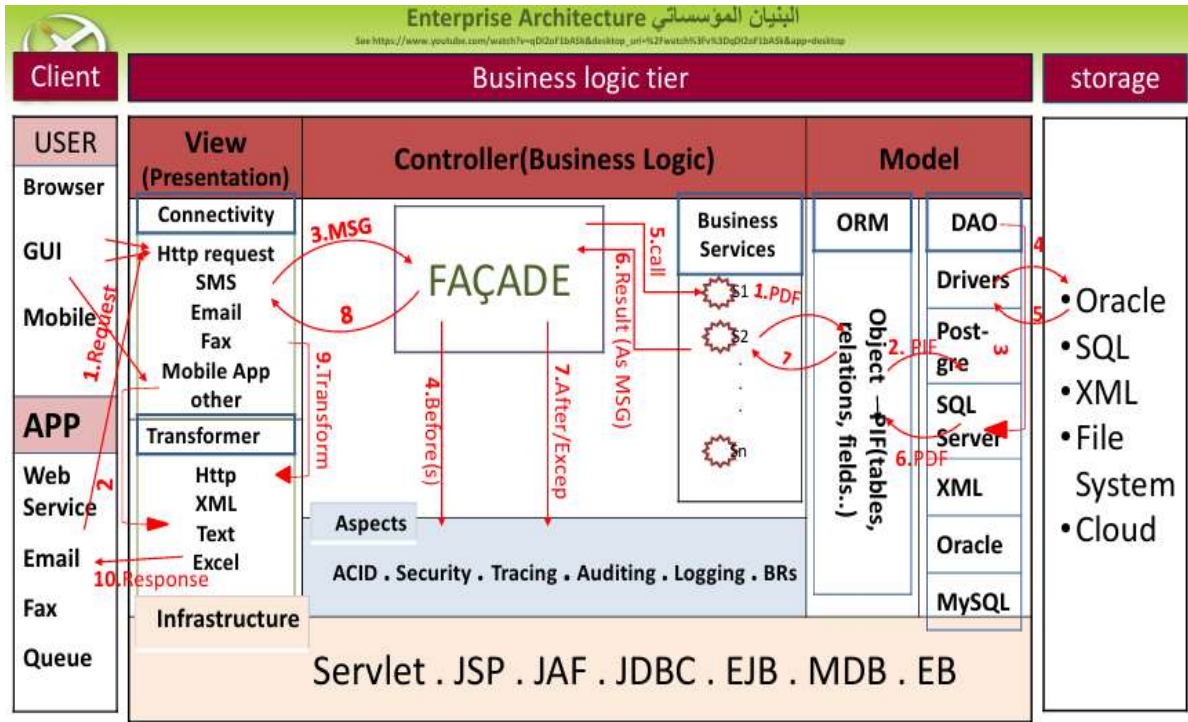
- إذن ال facade تستقبل الطلب و تحوله إلى الخدمة المناسبة .
- أهم ميزة لل Enterprise Architecture أنه يفصل بين المتطلبات الوظيفية و غير الوظيفية و يفصل بين المتطلبات الوظيفية و بعضها البعض و يفصل بين المتطلبات الغير وظيفية و بعضها البعض.
- يقوم ال facade بتطبيق مفهوم ال Aspect (تعرفنا عليها سابقا)، أمثلة عليها:
Acid – security – racing – auditing – logging – (Busines Rules) brs
- التطبيق الشبكي (التفرعي) يكون مكون من عدة خدمات موجودة في container للخدمات و كل خدمة مستقلة و قابلة لل deployment و النقل من سيرفر لآخر و التعديل بشكل مستقل عن الخدمات الأخرى.
- عندما يصل الطلب (message) إلى ال facade يقوم بإرسال طلب Before إلى Aspect و عند الانتهاء من الخدمة استدعي After / Excep لنفس ال Aspect، وإذا كان هنالك عدة Aspects على نفس الطلب يقوم ال facade باستدعائهم جميعا



AOP implementation Example

- Interface Aspect {
 - Msg before(Msg input);
 - Msg after(Msg result);
 - Msg onException(Exception ex);
- }

سؤال مهم: قم بإسقاط المستويات السبع على البنية المؤسساتية التالي:



:Façade Design Pattern

- تصل الـ Request من طبقة الزبون إلى طبقة presentation مباشرة إلى البوابة الموافقة: (http, SMS, Email, etc)
- تقوم هذه الطبقة باستعمال الـ transformer المناسب لتحويل الرسالة إلى Message وتضمن اسم الخدمة والمعاملات ونوع المناقشة والبوابة Port وإرسالها إلى الـ Facade
- تقوم Facade بـ:
 - تطبيق السمات القبلية before المناسبة عليها
 - توجيهها للخدمة الموافقة (بالاسم) عبر البوابة المطلوبة Middleware (EJB, Queue, Mail, etc)
 - تطبيق السمات البعدية after المناسبة عليها في حال نجاح الإرسال
 - تطبيق عمليات onException المناسبة في حال فشل الإرسال

مالذي يتغير من تطبيق لآخر؟

- نفس الـ instance => داتا بيز، مثال:
- نظام إدارة مسجد يتم استخدامه نفسه لكن في عدة مساجد
- غير instance => طبقة الخدمات، مثل:
- الفرق بين نظام إدارة مساجد أو نظام إدارة مشافي

سؤال مهم: هل يمكن لخدمة استدعاء خدمة أخرى دون المرور بال facade؟

لا يمكن ذلك لأن الـ facade هو المدخل الوحيد في مثل هذه التطبيقات المؤسساتية

سؤال مهم: ماذا نخسر في حال سمحنا للخدمات استخدام باقي الخدمات دون العبور عبر الـ facade؟

سنخسر جميع المتطلبات الغير وظيفية مثل الحماية – التتبع – الإنصات – توزيع الحمل

لماذا نخسر توزيع الحمل؟

لأن الـ FACADE عندما تأتيها خدمة تقوم بالبحث على سيرفر يستقبل الطلب، إذاً الـ FACADE تقوم بعملية الـ master slave.

فائدة أريسية:

- ماذا يعني master slave؟
تعني وجود سيرفر رئيسي master يتحكم ويوزع الطلبات على سيرفرات ثانوية slave ليتم تنفيذها

كيف تنتقل المعلومات إلى طبقة الـ storage؟

- عن طريق طبقة الـ model، و يوجد أسلوبين للتعبير عن المعلومات:
- formal: يمكن توصيفها للحاسب و بالتالي يمكن أتمتة هذه العملية.
- unformal: لا يمكن توضيح الآلية للحاسب، مثل التعرف على أجسام ضمن الصورة.

طبقة الـ model تتكون من:

1. ORM Design Pattern:

- اختصار لـ Object Relational Mapping
- يسمح بالانتقال آلياً من الأغراض في لغة البرمجة (جافا أو C++ أو غيرها) إلى شكل علاقتي يمكن تخزينه في قاعدة البيانات و بالعكس طبعاً.
- يساعد في توفير الوقت و الجهد و التركيز على البرمجة
- هذا يمثل الـ Object كيف مخزن بالداتابيز.
- الكود غير مرتبط فقط بهذه الداتابيز (أي يمكن تغييرها دون التأثير على العمل).
- بالإضافة أنها تساعد بتحقيق المتطلبات غير الوظيفية كـ platform independence لأن الـ ORM يمكننا من العمل على عدة قواعد معطيات Databases بنفس الكود وتحقيق Load Balancing لأن الكود سيعمل على عدة سيرفرات بدون مشاكل في تغيير السيرفر

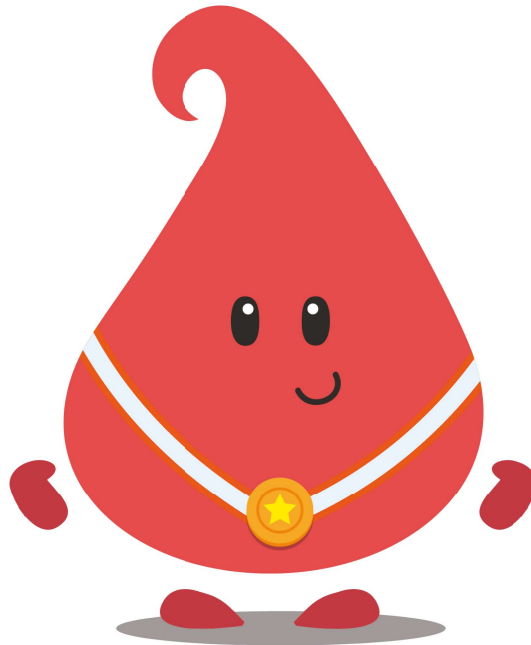


2. DAO Design Pattern :

- اختصار لـ Data Access Object
- تسمح بالانتقال الآلي من الترميز العلاقتي الناجم عن ORM إلى قاعدة البيانات و بالعكس.
- تسمح بالتعامل الشفاف مع قاعدة البيانات.
- تسمح بالتعامل مع عدة قواعد بيانات.
- تسمح بتحقيق الاستقلال عن قواعد البيانات.
- تقول وين موجودة الداتابيز.
- مستقل عن نوع الداتابيز

سيتم شرح الأسهم الحمراء و باقي الطبقات بشرح مفصل بالمحاضرات القادمة

ملاحظة: الأسئلة المهمة غالبا ستأتي بالامتحان لذلك يجب التركيز عليها و فهمها جيدا



**النجاح لا يقاس بالموقع الذي وصلت إليه
بل بالعقبات التي تغلبت عليها**